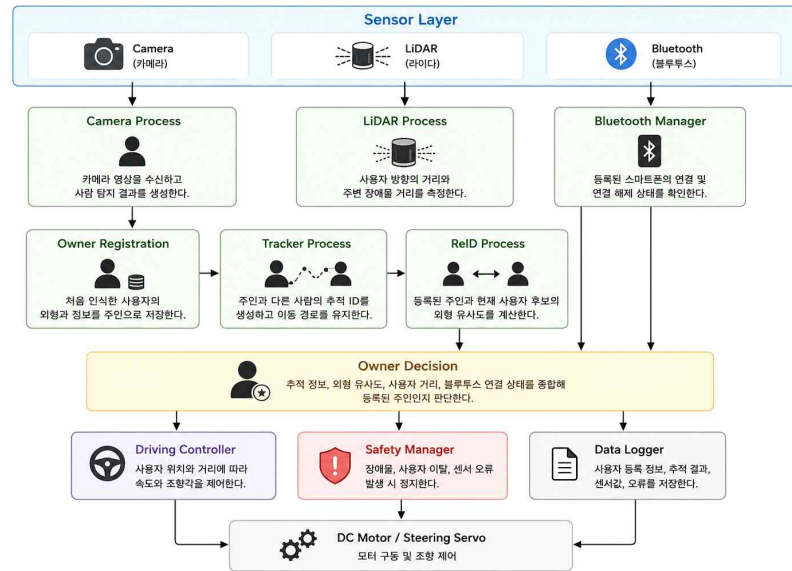


4조 프로젝트 기술서

프로젝트명: AutoCAR_Owner_Follow: 사용자 추종 시스템		
수행 기간	2026.07.13 ~ 2026.07.16 (4일)	
담당 역할	이연우	사용자 인식 알고리즘 설계, 카메라·LiDAR 센서 데이터 융합 기술 구현, 전체 프로젝트 일정 및 아키텍처 관리
	고태원	AIoT AutoCAR 모터 속도 제어, 조향 서보모터 각도 제어 알고리즘 및 주행 제어 펌웨어 구현
	장운호	Bluetooth 4.2 기반 스마트폰 연동, 사용자 초기 프로필 등록 프로세스 구축 및 JSON/SQLite 데이터 저장소 구현
	오시원	LiDAR 기반 장애물 감지, 사용자의 화면 이탈 시 PAN/TILT 카메라 재탐색 제어 및 다단계 안전 정지 모듈 구현
	김정연	다중 객체 탐지 및 추적 성능 시험 시나리오 작성, 교차 상황 및 인상착의 혼선 환경 테스트, 결과 데이터 로그 분석 및 발표 자료 정리
수행 목표	▪ 실시간 객체 탐지: AIoT AutoCAR Prime의 카메라 활용하여 첫 인식한 대상을 탐지하고 주인으로 등록 ▪ 다중 정보 기반 사용자 재식별: 상·하의 색상 및 무늬, 전체 외형 특징 벡터(OSNet ReID), 신체 비율 및 LiDAR 거리의 연속성을 종합 점수화하여 혼선 없는 사용자 식별 시스템 구현 ▪ 센서 융합 및 실시간 추종 주행: 카메라의 영상 좌표(방향 오차)를 LiDAR 각도로 변환·융합하여 1.3~1.7m의 일정한 목표 추종 거리 유지하는 조향 및 속도 제어 수행 ▪ 다중 예외 처리를 통한 안전성 확보: 장애물 감지, 사용자 이탈, Bluetooth 연결 해제, 주인 판단 불확실 상황 발생 시 즉시 감속 및 안전 정지 실행 ▪ 데이터 로깅 및 데이터베이스 구축: 사용자 등록 프로필 및 실시간 주행 상태 데이터, 센서 오류 정보를 JSON 또는 SQLite 기반의 시스템 로그로 안전하게 저장하고 관리	
사용 기술	구분	내용
	개발 하드웨어	AIoT AutoCAR Prime Board, DC 주행 모터, 조향 서보모터, PAN/TILT 서보모터
	센서 모듈	IMX219 광각 카메라, 360도 LiDAR, 내장형 Bluetooth 4.2 모듈
	운영체제	Soda OS (Ubuntu 기반 Linux 환경)
	개발 언어 및 도구	Python, Jupyter Notebook, VS Code, Git, GitHub, SSH 원격 접속
	영상 처리	OpenCV (영상 입력, 이미지 로그 및 실시간 프로세싱)
	객체 탐지 및 추적	경량 YOLO 모델 / MobileNet-SSD, DeepSORT / BoT-SORT
	재식별	OSNet 기반 ReID (외형 특징 벡터 추출)
	데이터 관리	JSON 형식 파일 저장 또는 SQLite 경량 데이터베이스
	주행 및 안전 제어	비례 제어 알고리즘(Kp), 데드존 필터, 다단계 조건문 기반 Safety Manager

세부 수행 내용	
상세 내용	1) 목적 : 본 프로젝트의 목적은 AutoCAR에 장착된 카메라, LiDAR 및 Bluetooth 기능을 이용하여 특정 사용자를 등록하고 자동으로 추종하는 시스템을 구현하는 것이다. 기존의 단순 사람 추종 방식은 영상에서 가장 가까운 사람이나 화면 중앙에 위치한 사람을 따라가기 때문에 여러 사람이 존재하거나 비슷한 옷을 입은 사람이 나타났을 때 추종 대상이 변경될 가능성이 있다. 이를 해결하기 위해 처음 인식한 사용자를 주인으로 등록하고, 이후 다음 정보를 종합하여 주인을 계속 구별한다. 2) 개발 환경 : - Python 기반 AutoCAR 제어 환경 - OpenCV 기반 카메라 영상 처리 - 경량 객체 탐지 모델을 이용한 사람 탐지 - DeepSORT 또는 BoT-SORT 기반 다중 객체 추적 - ReID 모델을 이용한 사용자 외형 특징 추출 - LiDAR 데이터를 이용한 사용자 거리 및 장애물 측정 - Bluetooth 기반 등록 스마트폰 연결 확인 - JSON 또는 SQLite 기반 사용자 프로필 및 실행 로그 저장 - Jupyter Notebook 또는 VS Code 기반 개발 - SSH를 이용한 AutoCAR 원격 접속 3) 주요 기능 : - AutoCAR 카메라 영상 실시간 입력 - 영상 속 사람 객체 탐지 - 사람별 경계 상자와 중심 좌표 계산 - 사람별 고유 추적 ID 부여 - 제스처를 수행한 사람을 주인으로 지정 - 사용자의 상의·하의·전체 외형 특징 저장 - 등록된 사용자의 이동 경로 추적 - 여러 사용자 후보의 주인 유사도 계산 - 카메라 좌표와 LiDAR 각도 연결 - 사용자와 AutoCAR 사이의 거리 측정 - 장애물 감지 및 긴급 정지 - Bluetooth 스마트폰 연결 상태 확인 - 사용자의 방향에 따른 조향각 제어 - 사용자 거리에 따른 주행 속도 제어 - 사용자 이탈 시 PAN 카메라 재탐색 - 주인 판단 불확실 시 정지 및 재인증 - 센서 오류 및 통신 오류 발생 시 안전 정지 - 탐지·추적·거리·주행 상태 로그 저장

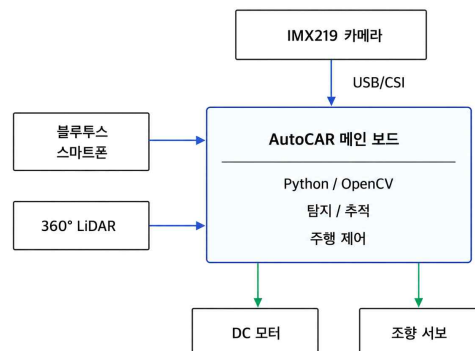
4) 시스템 구성 :



Camera Process	카메라 영상 수신하고 사람 탐지 결과 생성
Owner Registration	등록 사용자 외형과 추적 정보 저장
Tracker Process	사람별 추적 ID와 이동 경로 유지
ReID Process	등록 사용자와 현재 사용자 후보의 외형 유사도 계산
Bluetooth Manager	등록된 스마트폰의 연결 및 연결 해제 상태 확인
LiDAR Process	사용자 방향의 거리와 주변 장애물 거리 측정
Owner Decision	추적, 외형, 거리 정보를 종합해 주인 후보 판단
Driving Controller	사용자 위치와 거리에 따라 속도와 조향각 제어
Safety Manager	장애물, 사용자 이탈, 센서 오류 발생 시 정지
Data Logger	사용자 등록 정보, 추적 결과, 센서값, 오류 저장

5) 하드웨어 구성 :

IMX219 카메라	사람, 옷 및 외형 특징 인식
360도 LiDAR	사용자 거리와 장애물 위치 측정
Bluetooth	등록 스마트폰 연결 상태 확인
조향 서보	사용자의 좌우 위치에 따라 방향 조절
DC 모터	사용자 거리에 따라 주행 속도 조절
PAN/TILT 서보	사용자를 놓쳤을 때 카메라 재탐색



6) 핵심 코드 :

6-1) 사용자 특징 등록

```
def register_owner(  
    track_id: int,  
    appearance_features: list[float],  
    upper_features: list[float],  
    lower_features: list[float],  
    lidar_distance: float,  
) -> dict:  
    return {  
        "track_id": track_id,  
        "appearance_features": appearance_features,  
        "upper_features": upper_features,  
        "lower_features": lower_features,  
        "lidar_distance": lidar_distance,  
    }
```

여러 프레임에서 추출한 특징을 평균화하여 일시적인 자세와 조명 변화의 영향을 줄인다.

6-2) 등록 사용자 판단

```
def score(self, detection, previous_track_id, cds_value=None):  
    appearance = self._appearance(detection.feature, cds_value)  
    tracking = 1.0 if detection.track_id == previous_track_id else appearance * 0.6  
    clothing = appearance  
    body = 1.0  
    return (0.40 * tracking + 0.35 * appearance +  
            0.20 * clothing + 0.05 * body)
```

사용자가 등록된 주인인지 판단하기 위한 점수를 계산한다.

6-3) 주인 후보 점수 계산

```
def calculate_owner_score(candidate: dict) -> float:  
    return (  
        0.35 * candidate["tracking_continuity"]  
        + 0.30 * candidate["appearance_similarity"]  
        + 0.15 * candidate["clothing_similarity"]  
        + 0.15 * candidate["lidar_continuity"]  
        + 0.05 * candidate["body_similarity"]  
    )
```

외형만으로 판단하지 않고 이동 경로와 LiDAR 거리 연속성을 함께 적용한다.

6-4) 주인 판단

```
def classify_owner(  
    best_score: float,  
    second_score: float,  
) -> str:  
    if best_score < 0.70:  
        return "uncertain"  
  
    if best_score - second_score < 0.12:  
        return "ambiguous"  
  
    return "confirmed"
```

판단이 불확실하면 다른 사람을 자동으로 선택하지 않고 정지 상태로 전환한다.

플로우 차트	1. AutoCAR를 실행하고 카메라, LiDAR, Bluetooth를 초기화한다. 2. 등록된 스마트폰의 Bluetooth 연결을 확인하며, 연결되지 않은 경우 연결될 때까지 대기한다. 3. 카메라 활용하여 다중 사용자 중 30초 동안 6번 동안 탐지 & 4번 검증 후 주인으로 등록 사용자의 외형 특징과 추적 ID, LiDAR 거리 정보를 저장하여 주인 등록을 완료한다. 4. 사람 탐지 및 추적하면서 등록된 사용자인지 지속적으로 판단한다. 5. 주인 여부가 불확실한 경우 AutoCAR를 정지시키고 재인증을 요청한다. 6. LiDAR를 이용해 장애물을 확인하고, 장애물이 있으면 차량을 정지한 후 제거될 때까지 대기한다. 7. 사용자의 방향과 거리를 계산하여 적절한 속도와 조향각을 결정한다. 8. AutoCAR가 등록된 사용자를 추종하며, 사용자가 이탈하거나 Bluetooth 연결이 해제되면 다시 확인 과정을 수행하고 전체 과정을 반복한다.
참조	https://github.com/jungyeon220/AutoCar_Owner_Follow.git

